Università degli Studi di Verona

CORSO DI LAUREA IN INFORMATICA

Intelligenza Artificiale

Federico Brutti
federico.brutti@studenti.univr.it

Inserire citazione inerente alla materia

Indice

1 Agenti intelligenti **3**
1.1 Agenti e razionalità 3
1.2 Struttura degli agenti 4
1.3 Agenti Problem-solving 6

2 Risoluzione di problemi **8**
2.1 Ricerca nello spazio degli stati 8
2.2 Strategie di ricerca non informate 10
2.3 Strategie di ricerca informate 13
2.4 Ricerca locale 15
2.5 Soddisfacimento dei vincoli 15

3 Conoscenza, ragionamento e pianificazione **16**

4 Conoscenza incerta **17**

5 Apprendimento automatico **18**

Capitolo 1

Agenti intelligenti

1.1 Agenti e razionalità

Il campo dell'**intelligenza artificiale** ha lo scopo di comprendere e creare entità intelligenti, macchine capaci di calcolare come agire in modo efficace e sicuro in base alla situazione presa in esame. Si tratta di una disciplina molto ampia e suddivisa in un gran numero di sottoinsiemi; alcuni di questi, come l'apprendimento e la percezione, sono ambiti generali, mentre in altre diramazioni si applica l'AI anche per problemi specifici, come la dimostrazione di teoremi matematici. Esistono più scuole di pensiero riguardanti la definizione formale di "intelligenza"; una prima idea, che ha origine dalla psicologia, vede più intelligenti quelle entità capaci di rispecchiare al meglio la mente umana, ma tendenzialmente si preferisce usare la nozione di **razionalità**, ovvero, dato un problema particolare, di compiere un'azione corretta per avvicinarsi alla sua risoluzione.

Le entità usate in questa disciplina si chiamano **Agenti**, i quali rappresentano un sistema capace di percepire il suo ambiente attraverso dei **sensori** e agisce su di esso mediante **attuatori**; non dissimilmente da come una persona percepisce e interagisce con ciò che lo circonda. In particolare, i dati percepiti dagli agenti formano una **sequenza percettiva**, ovvero lo storico di quanto da essi osservato all'interno del loro ambiente. Il comportamento di un agente è poi descritto da una **funzione agente**, la quale mostra la corrispondenza tra una qualsiasi sequenza percettiva e una specifica azione, ed è implementata dal **programma agente**, un algoritmo in esecuzione all'interno del sistema fisico.

Potendo inserire diversi algoritmi usati come programma agente, è necessario capire in che modo determinare quando un agente è adatto al compito che gli è richiesto di eseguire. Richiamando il precedente concetto, definiamo **agenti razionali** quei sistemi che eseguono un'azione "corretta", atta alla risoluzione del problema, la quale è valutata in base a una **misura di prestazione**, il criterio per definire quanto efficaci siano state. In questi termini, è auspicabile massimizzare il valore atteso da questa misura per ogni

sequenza percettiva ottenibile, mostrando come la qualità di un agente sia un problema di ottimizzazione.

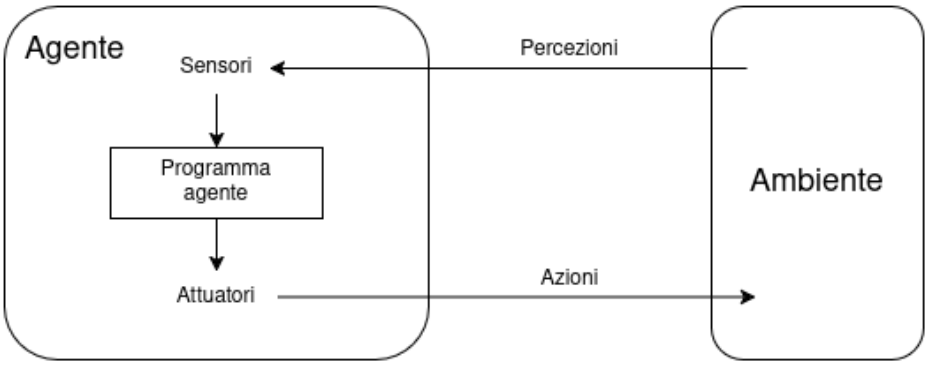


Figura 1.1: Struttura di un agente

Supponiamo per esempio un ambiente composto da due stanze sporche e l'obiettivo è pulirle tutte. Si potrebbe usare il seguente programma agente per rimuovere lo sporco:

Algorithm 1 Reflex-Vacuum-Agent(location, status) **returns** action

```
if status = dirty then return Suck
else if location = A then return Right
else if location = B then return Left
end if
```

Se l'unica misura di prestazione è la pulizia dell'ambiente, possiamo dire che questo agente è razionale e buono, ma se si aggiungessero ulteriori condizioni sarà necessario rivedere l'algoritmo utilizzato. È comune, in problemi come questo, usare come misura il valore di **average idleness**, la somma degli intervalli di tempo per i quali l'agente non visita altri stati, per poterla minimizzare e risolvere il problema il prima possibile. In questi termini è desiderabile considerare tutti gli stati allo stesso modo, con lo scopo di ottenere valori medi i più omogenei possibile. Ciò si traduce nel cercare di minimizzare il valore di varianza, siccome più basso sarà, tanto più omogenei saranno i valori di idleness.

1.2 Struttura degli agenti

Ora che abbiamo compreso le componenti che formano gli agenti razionali, è necessario fornire una visione più formale di cosa definire per poterli progettare al meglio. Gli elementi sono dati dall'acronimo **PEAS**, ovvero:

- **Performance measure**: Risultato desiderato dall'agente.

- **Environment**: Ambiente di lavoro al quale è applicato l'agente.
- **Actuators**: Elementi che definiscono come l'agente può compiere azioni.
- **Sensors**: Elementi che definiscono come l'agente è in grado di percepire l'ambiente.

Esistono inoltre diversi tipi di ambienti con una struttura condivisa per poter definire il tipo di problema preso in esame. È di fondamentale importanza conoscerli a fondo poiché il design degli algoritmi si baserà in gran parte su di essi:

- **Osservabile**: Lo stato del mondo è completamente visibile dall'agente.
- **Deterministico**: Le azioni hanno conseguenze perfettamente predicibili dall'agente.
- **Episodico**: Le azioni che l'agente compie impattano esclusivamente la sola istanza di lavoro corrente. Ciò non fa cambiare strategia.
- **Statico**: Si lavora sempre nello stesso ambiente non modificabile; solo le azioni dell'agente potranno cambiare.
- **Discreto**: Il problema è esprimibile con sole variabili discrete.
- **A singolo agente**: Il problema presenta l'utilizzo di un singolo agente.

In base al tipo di problema proposto, si tenderà a cadere in uno dei seguenti quattro tipi di agenti:

- **Simple-reflex**: Crea una risposta in base alla percezione corrente e le condizioni interne.
- **Reflex with state**: Come il precedente, ma mantiene anche uno stato interno. Comunemente utilizzato quando la conoscenza dell'ambiente non è sufficiente a determinare una risposta.
- **Goal-based**: Mantiene un obiettivo finale da raggiungere e presenterà un blocchetto di codice in più per osservare come il mondo si evolve in base alle proprie azioni. Richiede un environment completamente osservabile e deterministico per funzionare.
- **Utility-based**: Fornisce un valore di utilità a ogni stato e il suo scopo è massimizzare quanto ottenuto.

Ognuna di queste quattro categorie di agente ha la possibilità di diventare **intelligente**, per i quali è assegnato un critico il cui compito è dire all'agente quanto bene sta performando in base alla misura di prestazione stabilita.

1.3 Agenti Problem-solving

Se consideriamo gli agenti goal-based, ovvero basati su obiettivi, è possibile delinearne un sottoinsieme composto da quelli capaci di **risolvere problemi**, i quali formulano un obiettivo e un problema dato lo stato corrente, per cercare una soluzione ed eseguirla. Le loro variabili che compongono l'ambiente vengono quindi inizializzate una sola volta, all'inizio, e sono:

- **seq**: Sequenza di azioni compiute dall'agente.
- **state**: Descrizione dello stato corrente dell'ambiente.
- **goal**: Obiettivo definito all'avvio.
- **problem**: Formulazione del problema. visto come quadrupla di stato iniziale, funzione successore, goal test e path cost.

In particolare, i problemi sono definiti da una quadrupla composta da **stato iniziale**, **funzione successore**, la quale a sua volta è un insieme di coppie (azione, stato) $S(A) = \{\langle state_1 \rightarrow state_2 \rangle, ...\}$, **goal test**, indicante l'obiettivo, e il **costo del percorso**, un valore additivo dato per esempio dalla somma del costo di attraversamento degli archi in un grafo.

Dunque, all'avvio, quando *seq* è vuota, si formulano *goal* e *problem* e si attua il processo di ricerca sul problema per poter determinare la prima azione da compiere e aggiungere alla sequenza. Dopo questa prima iterazione, si riparte e l'agente aggiornerà lo stato ricevendo una nuova percezione. Il processo continua fino al termine della ricerca. In pseudocodice:

Algorithm 2 Simple-Problem-Solving-Agent(percept) **returns** action

```
state ← Update-State(state, percept)
if seq is empty then
    goal ← Formulate-Goal(state)
    problem ← Formulate-Problem(state, goal)
    seq ← Search(problem)
end if
action ← First(seq)
seq ← Rest(seq)
return action
```

Questo è rappresentativo di un agente basato su obiettivi applicato in un ambiente deterministico e completamente osservabile, ma non sempre è possibile conoscere tutto; in tal caso si avrebbe un problema **conformant**, dove l'agente sarebbe cieco e non avrebbe modo di capire dove si trovi, né se sta lavorando bene.

Se invece i problemi non sono deterministici o parzialmente osservabili, si ha un problema di **contingenza**, nel quale le percezioni forniscono informazioni nuove in base allo stato corrente e il problema dovrà essere risolto con un piano di contingenza.

Infine, se lo spazio degli stati è del tutto socnosciuto, si ha un problema di **esplorazione** e per risolverlo sarà necessario usare quello che è conosciuto come reinforcement learning.

Capitolo 2

Risoluzione di problemi

2.1 Ricerca nello spazio degli stati

Può capitare che l'azione giusta da compiere non risulti subito evidente e che un agente, per poterla determinare, abbia la necessità di guardare in avanti, ossia considerare una sequenza di azioni che lo porti a uno stato obiettivo. Gli agenti risolutori di problemi, menzionati nel precedente capitolo, svolgono esattamente questo compito e fanno uso di rappresentazioni atomiche per gli stati, i quali sono privi di una struttura interna visibile al programma agente. La ricerca della sequenza consiste nel generare i successori degli stati già osservati, una procedura chiamata **espansione degli stati**, e un primo approccio è data dall'algoritmo **Tree-Search**, la cui idea è:

Algorithm 3 Abstract-Tree-Search(problem, strategy) **returns** solution or failure

```
initialize search tree using initial state of problem
loop
    if No candidates for expansion then return failure
    end if
    Choose a leaf node for expansion according to strategy
    if Node contains a goal state then return solution
    else Add successor nodes to search tree
    end if
end loop
```

Gli algoritmi usati per la ricerca all'interno degli alberi sono **Breadth-First-Search** e **Depth-First-Search**, insieme ai quali è anche creata la **frontiera**, una coda che segue la logica FIFO se è applicato BFS, o LIFO con DFS; l'ordine varia in base alla strategia. Fondamentalmente è quest'ultima che contiene la sequenza di azioni da attuare, la quale ritornerà un successo se termina con una soluzione, altrimenti l'agente riporterà un fal-

limento. Per poter utilizzare la sequenza data dalla frontiera è necessario prelevare un nodo per iterazione chiamando il metodo **Pop()**, nel seguente modo:

Algorithm 4 Tree-Search(problem, frontier) **return** solution or failure

```
frontier ← Insert(Make-Node(problem.Initial-State))
while ¬IsEmpty(frontier) do
    node ← Pop(frontier)
    if problem.Goal-Test(node.State) then return node
    end if
    frontier ← InsertAll(Expand(node, problem))
end while
return failure
```

Dove il metodo **Expand()** è definito come segue:

Algorithm 5 Expand(node, problem) **returns** set of nodes

```
successors ← ∅
for each action, result in Successor-Fn(problem, node.State) do
    s ← new Node
    s.Parent-Node ← node
    s.Action ← action
    s.State ← result
    s.Path-Cost ← node.Path-Cost + Step-Cost(node.State, action, result)
    s.Depth ← node.Depth+1
    add s to successors
end for
return successors
```

Le strategie di risoluzione sono valutate in più ambiti: **completezza**, capace di trovare sempre una soluzione se esiste, **complessità temporale**, il numero di nodi generati o espansi, **complessità spaziale**, il numero massimo di nodi in memoria, e **ottimalità**, la capacità di trovare sempre la soluzione di costo minimo. In particolare, le due complessità sono a loro volta misurate in termini di:

- b: Massimo numero di figli che un nodo può avere all'interno dell'albero di ricerca.
- d: Profondità della soluzione di costo minimo.
- m: Profondità massima dello spazio degli stati (possibile che sia infinita).

Nell'ambito dello spazio degli stati, non scoprire che alcuni stati sono ripetuti potrebbe trasformare un problema di complessità lineare in uno esponenziale. Si tratta di un

problema evitabile aggiungendo una struttura contenente gli stati già esplorati, una soluzione non dissimile a quella degli RB alberi. **Graph-Search()** è l'algoritmo che implementa questa caratteristica:

Algorithm 6 Graph-Search(problem, frontier) **return** solution or failure

```
explored ← ∅, frontier ← Insert(Make-Node(problem.Initial-State))
while ¬IsEmpty(frontier) do
    node ← Pop(frontier)
    if problem.Goal-Test(node.State) then return node
    end if
    if node.State ∉ explored then
        add node.State to explored
        frontier ← InsertAll(Expand(node, problem))
    end if
end while
return failure
```

2.2 Strategie di ricerca non informate

Le strategie di ricerca non informate sono approcci di ricerca che fanno uso esclusivamente delle informazioni disponibili nella definizione del problema. Sono attuabili con più algoritmi, tra cui i seguenti:

Breadth-First-Search: Attua un'espansione sul nodo più superficiale non ancora espanso. La frontiera è implementata come una coda FIFO.
L'algoritmo è **completo** in quanto b ha valore finito, presenta una **complessità temporale** e **spaziale** di $O(b^d)$ poiché il massimo numero di figli che un nodo ha cresce esponenzialmente andando verso le foglie, i quali sono tenuti in memoria. Dato questo salvataggio, è sconsigliato implementarlo quando si ha un grande numero di nodi. Inoltre, è **ottimale** solamente quando il costo per ogni step è uguale, qunidi non in termini generali.

Uniform-Cost-Search: Algoritmo greedy che espande il nodo il cui step ha costo minore. Si tratta di una rielaborazione di Dijkstra specializzato nella ricerca. Per implementarlo è richiesta una frontiera data da una coda che ordina i path-cost in modo crescente. Inoltre, rispetto a BFS, è consigliato applicare due modifiche per garantire ottimalità: chiamare il goal-test *durante* la selezione del nodo per l'espansione e controllare se un nodo figlio sia già presente nella frontiera con un costo maggiore; in tal caso, bisognerà rimpiazzarlo con quello appena trovato. Inoltre:

Algorithm 7 BFS(problem) **returns** solution or failure

```
node ← node with State = problem.Initial-State, Path-Cost = 0
if problem.Goal-Test(node.State) then return node
end if
explored ← ∅, frontier ← queue with single node element
loop
    if frontier = ∅ then return failure
    end if
    add node.State to explored
    for each action in problem.Actions(node.State) do
        child ← Child-Node(problem, node, action)
        if child.State ∉ explored or frontier then
            if problem.Goal-Test(child.State) then return child
            end if
            frontier ← insert(child)
        end if
    end for
end loop
```

- **Completo** se lo step cost è $\geq \epsilon$.
- **Complessità temporale**: Data dal numero di nodi con $g \leq$ costo della soluzione ottimale C^* : $O(b^{1+\lfloor C^*/\epsilon \rfloor})$.
- **Complessità spaziale**: Data dal numero di nodi con $g \leq$ costo della soluzione ottimale C^* : $O(b^{1+\lfloor C^*/\epsilon \rfloor})$.
- **Ottimale** poiché i nodi si espandono per l'ordine crescente di $g(n)$.

Depth-First-Search: Espande il nodo di profondità maggiore non ancora espanso. Richiede una frontiera implementata con coda LIFO. Inoltre:

- **Non completo**: Fallisce quando si ha una profondità infinita oppure trova dei cicli infiniti.
- **Complessità temporale**: $O(b^m)$; Atroce quando la profondità massima dello spazio degli stati m è molto maggiore di d, ovvero quella della soluzione di costo minimo. Tuttavia, se le soluzioni sono dense, potrebbe essere più veloce di BFS.
- **Complessità spaziale**: $O(bm)$, lineare.
- **Non ottimale**.

Iterative-Deepening-Search: Strategia che ritorna una soluzione chiamando una sotto-procedura chiamata **Depth-Limited Search**, basata sulla logica di DFS ma aggiungendo uno specifico limite di profondità, sotto al quale non controllerà i nodi:

Algorithm 8 IDS(problem) **returns** solution

```
for depth ← 0 to ∞ do
    result ← DLS(problem, depth)
    if result ≠ cutoff then return result
    end if
end for
```

Notare la chiamata a DLS, la quale fa uso di una chiamata a **Recursive-DLS()**, un'ulteriore procedura ricorsiva. Le implementazioni in pseudocodice sono:

Algorithm 9 DLS(problem, limit) **returns** solution, failure or cutoff

```
R-DLS(Make-Node(problem.Initial-State), problem, limit)
```

Algorithm 10 R-DLS(node, problem, limit) **returns** solution, failure or cutoff

```
if problem.Goal-Test(node.State) then return node
else if limit = 0 then return cutoff
else
    cutoff-occurred ← false
    for each action in problem.Actions(node.State) do
        child ← Child-Node(problem, node, action)
        result ← R-DLS(child, problem, limit-1)
        if result = cutoff then cutoff-occurred ← true
        else if result ≠ failure then return result
        end if
    end for
    if cutoff-occurred then return cutoff
    else return failure
    end if
end if
```

Questa strategie propone un algoritmo **completo**, con una **complessità temporale** di $O(b^d)$ e **spaziale** di $O(bd)$. È anche ottimale se lo step cost è uguale a uno; inoltre, può essere modificato per esplorare alberi di costo uniforme.

2.3 Strategie di ricerca informate

Le strategie di ricerca informate sfruttano conoscenze specifiche del dominio applicativo per fornire suggerimenti, i quali prendono la forma di una **funzione euristica** $h(n)$, su dove si possa trovare l'obiettivo. La funzione rappresenta il costo stimato del cammino di costo minimo dallo stato del nodo n a quello di un obiettivo. Inoltre, risultano in generale più efficienti rispetto a strategie non informate.

Greedy Best-First search: Sfrutta la funzione euristica $h(n)$ su ogni nodo per determinare quale riporti il valore più basso, quindi più vicino all'obiettivo, per poi espanderlo. La frontiera è implementata come una coda di ordine crescente sul valore delle $h(n)$ calcolate.

Dunque, partendo dallo stato iniziale, si effettua la sua espansione, per poi scegliere il passo seguente in base ai path cost dei nodi successivi. Si continua a scegliere quello di costo minimo fino al raggiungimento dell'obiettivo.

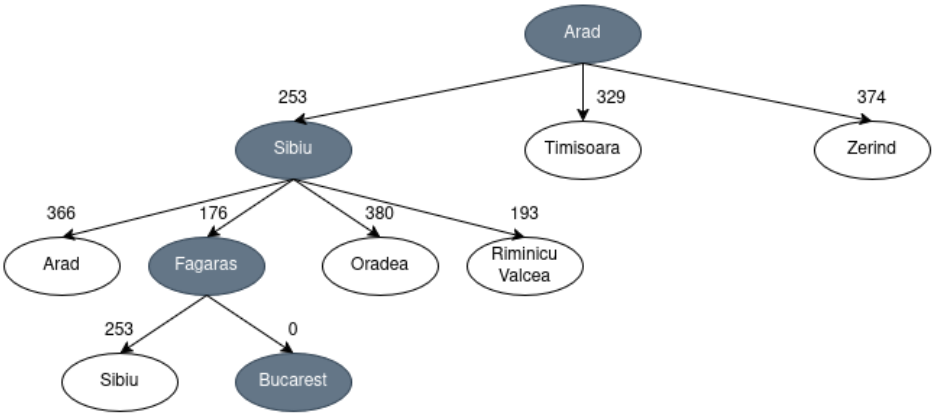


Figura 2.1: Esempio di Greedy best first search

L'algoritmo presenta poi le seguenti caratteristiche:

- **Non completo**: Potrebbe rimanere bloccato in cicli, ma può diventarlo in uno spazio finito con un controllo su stati ripetuti.
- **Complessità temporale**: $O(b^m)$, tuttavia una buona euristica può fornire buoni miglioramenti.
- **Complessità spaziale**: $O(b^m)$, perché mantiene tutti i nodi in memoria
- **Non ottimale**.

A* search: Altro algoritmo basato sull'approccio best-first la cui idea è di evitare l'espansione di cammini più dispendiosi rispetto a quello minimo trovato. Sfrutta la funzione di valutazione:

$$f(n) = g(n) + h(n)$$

Dove $g(n)$ è il costo del cammino dal nodo iniziale ad n ed $h(n)$ è il costo stimato per raggiungere l'obiettivo partendo da n. Ciò rende $f(n)$ il costo stimato del cammino migliore che continua da n fino a un obiettivo. Inoltre, A* fa uso di un'euristica ammissibile, la quale sarà definita meglio più avanti.

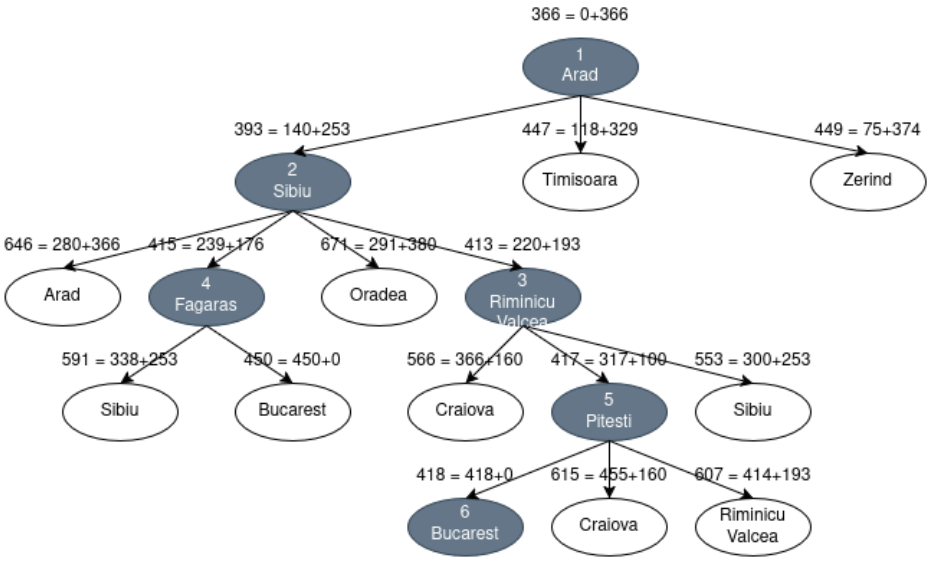


Figura 2.2: Esempio di A* search

L'algoritmo presenta le seguenti caratteristiche:

- **Completo**, a meno che non ci siano infinitamente tanti nodi con $f \leq f(G)$
- **Complessità temporale**: Esponenziale in [err. relativo in h $\times$ lunghezza sol.]
- **Complessità spaziale**: Mantiene tutti i nodi in memoria.
- **Ottimale**, ma richiede di fare assunzioni sull'euristica, come l'ammissibilità e la consistenza, e dipende dalla strategia di ricerca.

Definiamo ora un'euristica **ammissibile** se non sovrastima mai il costo reale $h^*(n)$ per arrivare all'obiettivo a partire da n; nel caso in cui sbagli, potrà essere solo per difetto e mai per eccesso. Rispetta la relazione:

$$h(n) \leq h^*(n); \quad h(n) \geq 0; \quad h(G) = 0 \text{ con G obiettivo}$$

Una condizione più forte è fornita dalle euristiche **consistenti**, per le quali la stima da n non può essere peggiore della stima da un nodo n', successore di n, sommata al costo reale per andare da n a n'. Si tratta quindi di una disuguaglianza triangolare applicata al grafo di ricerca; formalmente:

$$h(n) \leq c(n, a, n') + h(n'); \quad a \text{ azione che genera } n'$$

La conseguenza pratica di questa condizione è che i valori $f(n)$ di A* non decrescono mai lungo un cammino, garantendo anche l'ottimalità trovando la soluzione di costo minimo. Inoltre ogni euristica consistente è anche ammissibile, ma non viceversa.

Se consideriamo ora due euristiche ammissibili h_1, h_2, e per ogni nodo vale $h_2(n) \geq h_1(n)$, allora si dice che h_2 **domina** h_1, e, poiché più informata, risulterà migliore rispetto all'altra. La dominanza fornisce quindi un criterio per scegliere l'euristica migliore applicata al caso in esame.

Per costruire euristiche ammissibili ci si affida ai **problemi rilassati**; contesti nei quali si prende il problema originale e si rimuovono (rilassano) alcuni vincoli sulle azioni, rendendo il problema più semplice da risolvere. Il costo della soluzione ottima di questo nuovo problema risulta essere un'euristica ammissibile anche per l'originale, perché ogni soluzione di quest'ultimo è valida anche sui problemi rilassati, il cui costo non supererà mai quello vero.

2.4 Ricerca locale

2.5 Soddisfacimento dei vincoli

Capitolo 3

Conoscenza, ragionamento e pianificazione

agenti basati sulla logica; rappresentazione della conoscenza; pianificazione automatica;

Capitolo 4

Conoscenza incerta

quantificare l'incertezza; ragionamento probabilistico; problemi di decisione sequenziali;

Capitolo 5

Apprendimento automatico

apprendimento da esempi, apprendimento per rinforzo, basi per IA generativa.